

PCS-302 : Data Structures Lab

Circular Queue | Create, EnQueue, DeQueue, IsEmpty, Front, Rear

Problem Statement

Design an algorithm and write a program to implement it. A circular queue is a queue in which the last position of the queue is connected with the first position of the queue to make a circle. The program should implement the following operations: Create(), EnQueue(k), DeQueue(), IsEmpty(), Front(), Rear().

Example

MAX = 5, indices wrap around using % MAX:

```
enqueue(10) --> [10]                front=0, rear=0
enqueue(20) --> [10,20]              rear=1
enqueue(30) --> [10,20,30]           rear=2
enqueue(40) --> [10,20,30,40]        rear=3
enqueue(50) --> [10,20,30,40,50]     rear=4 (queue is now FULL)

dequeue()   --> removes 10            front=1

enqueue(60) --> rear wraps: (4+1)%5 = 0  arr[0]=60
               Queue is now: 20 30 40 50 60 (circular!)

Front() --> 20
Rear()  --> 60
```

Algorithm (Circular Array Using % MAX)

```
Create():
    front = -1, rear = -1
```

```
IsEmpty():
    return (front == -1)
```

```
IsFull():
    return ((rear + 1) % MAX == front)
```

```
EnQueue(k):
    if IsFull(): print "Overflow", return
    if IsEmpty(): front = 0, rear = 0
    else: rear = (rear + 1) % MAX
    arr[rear] = k
```

```
DeQueue():
    if IsEmpty(): print "Underflow", return
    remove arr[front]
    if front == rear: front = -1, rear = -1 // last element removed
    else: front = (front + 1) % MAX
```

```
Front():
    if IsEmpty(): print "Empty"
```

```
        else: return arr[front]
```

```
Rear():  
    if IsEmpty(): print "Empty"  
    else: return arr[rear]
```

C Program

```
#include <stdio.h>  
#define MAX 5  
  
int arr[MAX];  
int front = -1, rear = -1;  
  
void create() {  
    front = -1;  
    rear = -1;  
    printf("Circular queue created.\n");  
}  
  
int isEmpty() {  
    if (front == -1)  
        return 1;  
    else  
        return 0;  
}  
  
int isFull() {  
    if ((rear + 1) % MAX == front)  
        return 1;  
    else  
        return 0;  
}  
  
void enqueue(int k) {  
    if (isFull() == 1) {  
        printf("Queue Overflow. Cannot enqueue %d.\n", k);  
        return;  
    }  
    if (isEmpty() == 1) {  
        front = 0;  
        rear = 0;  
    } else {  
        rear = (rear + 1) % MAX;  
    }  
    arr[rear] = k;  
    printf("%d enqueued to queue.\n", k);  
}  
  
void dequeue() {  
    if (isEmpty() == 1) {  
        printf("Queue Underflow. Cannot dequeue.\n");  
        return;  
    }  
    printf("%d dequeued from queue.\n", arr[front]);
```

```

    if (front == rear) {
        front = -1;        // queue becomes empty
        rear = -1;
    } else {
        front = (front + 1) % MAX;
    }
}

void frontElement() {
    if (isEmpty() == 1) {
        printf("Queue is empty.\n");
        return;
    }
    printf("Front element = %d\n", arr[front]);
}

void rearElement() {
    if (isEmpty() == 1) {
        printf("Queue is empty.\n");
        return;
    }
    printf("Rear element = %d\n", arr[rear]);
}

int main() {
    create();

    enqueue(10);
    enqueue(20);
    enqueue(30);
    enqueue(40);
    enqueue(50);

    frontElement();
    rearElement();

    dequeue();
    enqueue(60);        // wraps around to index 0

    frontElement();
    rearElement();

    return 0;
}

```